

AEC3 – Análisis de Redes e Integración de LLMs: Arquitectura Inteligente para Análisis de Datos

🕒 Creado	31 de mayo de 2026 22:55
🏷️ Etiquetas	



1. Introducción

Este proyecto representa la evolución final del pipeline iniciado en AEC1 y ampliado en AEC2. Mientras que las entregas anteriores estaban orientadas principalmente a la extracción, limpieza y análisis descriptivo de datos, esta iteración incorpora una arquitectura orientada al análisis estructural y razonamiento asistido mediante inteligencia artificial.

El objetivo principal consiste en transformar datos no estructurados procedentes de redes sociales en conocimiento interpretable mediante una combinación de:

- Ingeniería de datos modular.
- Procesamiento de lenguaje natural.
- Teoría de grafos.
- Modelos de lenguaje ejecutados localmente.

La propuesta no se limita únicamente a obtener estadísticas o visualizaciones. El sistema busca responder preguntas más complejas:

- ¿Qué usuarios poseen mayor capacidad de influencia?
- ¿Qué comunidades aparecen dentro de una red?
- ¿Qué relaciones existen entre distintos grupos?
- ¿Cómo puede un modelo de lenguaje interpretar esas estructuras?

Para resolver estas cuestiones, el proyecto incorpora una capa de análisis semántico basada en modelos LLM ejecutados localmente mediante Ollama.

2. Objetivos del proyecto

Los objetivos planteados para esta entrega son:

Objetivos funcionales

- Centralizar toda la lógica de negocio mediante una arquitectura modular.
- Construir grafos de interacción a partir de datos sociales.
- Detectar usuarios relevantes mediante métricas de centralidad.
- Identificar comunidades automáticamente.
- Integrar un modelo de lenguaje local capaz de interpretar resultados.
- Proporcionar una interfaz visual interactiva mediante Streamlit.

Objetivos técnicos

- Diseñar un sistema mantenible y escalable.
- Reducir dependencias externas.
- Garantizar privacidad mediante ejecución local.
- Permitir reutilización futura del código desarrollado.

3. Arquitectura general del sistema

La arquitectura ha sido rediseñada respecto a las entregas anteriores siguiendo principios de separación de responsabilidades.

La clase principal `DataExtractor` deja de asumir toda la lógica y pasa a funcionar como orquestador del sistema.

```
AEC3/
├── dashboard/
│   └── app.py
├── data/
│   ├── raw/
│   ├── processed/
│   └── output/
├── src/
│   ├── __init__.py
│   ├── data_extractor.py
│   ├── network_core.py
│   ├── llm_core.py
│   └── nlp_core.py
├── AEC3_DataExtractor.ipynb
├── requirements.txt
├── .env.example
└── README.md
```

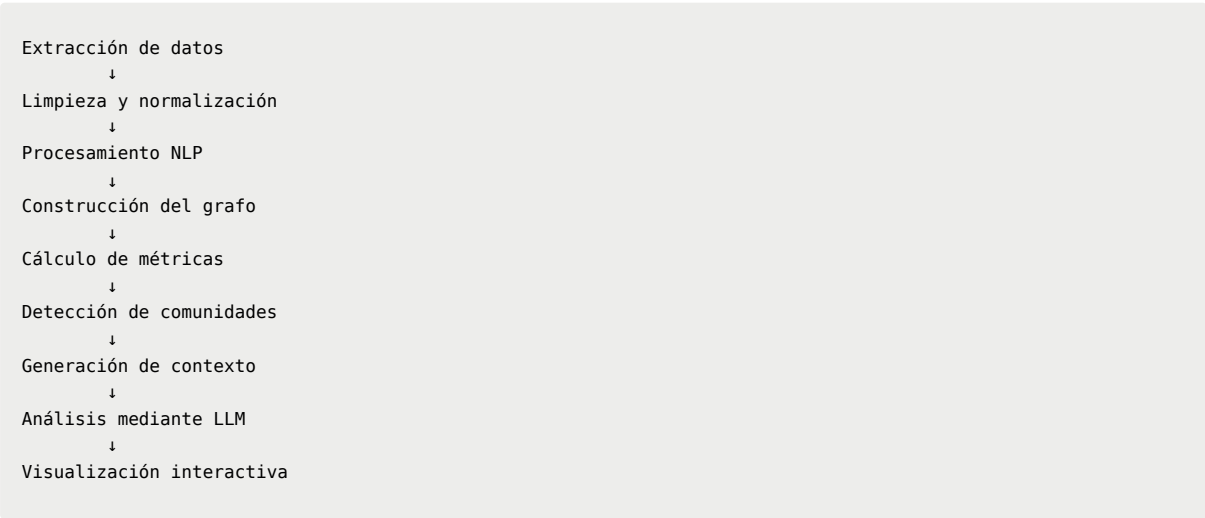
Cada módulo posee una responsabilidad específica:

Módulo	Responsabilidad
<code>data_extractor.py</code>	Orquestación global del pipeline
<code>nlp_core.py</code>	Limpieza y procesamiento lingüístico
<code>network_core.py</code>	Construcción y análisis de grafos
<code>llm_core.py</code>	Comunicación con Ollama
<code>dashboard</code>	Interfaz interactiva

Esta organización mejora la mantenibilidad y reduce el acoplamiento entre componentes.

4. Flujo metodológico

El procesamiento sigue una secuencia estructurada:



Cada etapa produce información utilizada por la siguiente, permitiendo construir progresivamente conocimiento a partir de datos inicialmente no estructurados.

5. Construcción y análisis de redes

Uno de los elementos centrales de AEC3 consiste en representar las interacciones sociales como grafos dirigidos.

Definición utilizada:

- **Nodo** → usuario.
- **Arista** → mención entre usuarios.
- **Peso** → frecuencia de interacción.

La representación mediante grafos permite aplicar métricas de teoría de redes imposibles de obtener mediante análisis tradicionales.

Métricas implementadas:

Métrica	Interpretación
In-degree	Número de menciones recibidas
Out-degree	Número de menciones realizadas
Betweenness	Capacidad de actuar como puente
PageRank	Influencia global relativa
Louvain	Detección de comunidades

Estas métricas permiten identificar:

- líderes de opinión;
- usuarios puente;

- comunidades altamente conectadas;
 - patrones de comportamiento colectivos.
-

6. Integración de Inteligencia Artificial Generativa

La principal evolución respecto a las entregas anteriores consiste en incorporar una capa de razonamiento contextual mediante modelos de lenguaje (LLMs). Hasta este punto, el pipeline era capaz de extraer información, transformarla y generar métricas descriptivas; sin embargo, seguía existiendo una limitación importante: los resultados cuantitativos requerían interpretación manual.

Por ejemplo, detectar que un usuario presenta un PageRank elevado o que una comunidad posee una alta densidad de conexiones aporta información relevante, pero no explica necesariamente qué significa dicho comportamiento dentro del contexto social analizado.

Para resolver esta limitación se introduce un modelo de lenguaje local capaz de actuar como una capa interpretativa adicional.

El flujo implementado sigue la siguiente lógica:

1. Se calculan métricas estructurales sobre el grafo.
2. Los resultados se condensan en un contexto semántico.
3. El contexto generado se transforma en un prompt estructurado.
4. El modelo procesa esa información.
5. Se obtiene una explicación contextual de los patrones encontrados.

La finalidad no es sustituir algoritmos clásicos mediante inteligencia artificial, sino combinar ambos enfoques.

Los algoritmos tradicionales proporcionan precisión matemática y capacidad analítica. El modelo de lenguaje añade interpretación, contextualización y capacidad explicativa.

Este enfoque híbrido permite pasar de una simple descripción estadística a una comprensión más cercana a un proceso real de análisis y toma de decisiones.

7. Justificación tecnológica: elección de Ollama frente a Hugging Face

Uno de los requisitos propuestos para esta entrega consistía en incorporar modelos de lenguaje mediante Hugging Face. Sin embargo, tras analizar distintas alternativas, decidí utilizar Ollama como capa de inferencia local manteniendo la misma filosofía funcional, pero modificando la forma de integración.

Esta decisión no responde a una incompatibilidad con Hugging Face ni a una limitación técnica; de hecho, Hugging Face constituye actualmente uno de los ecosistemas más importantes dentro del desarrollo de aplicaciones basadas en inteligencia artificial. Su catálogo de modelos y herramientas resulta extremadamente amplio y flexible.

La elección se realizó principalmente por cuestiones relacionadas con arquitectura, mantenibilidad y experiencia práctica de desarrollo.

Simplificación de la gestión de modelos

Una integración tradicional mediante Hugging Face suele requerir gestionar múltiples elementos:

- descarga manual del modelo;
- carga de tokenizadores;
- gestión de precisión numérica;
- asignación de memoria;
- configuración de GPU;
- optimizaciones de inferencia.

Aunque este enfoque proporciona un alto nivel de control, introduce una complejidad adicional que no aporta valor directo a los objetivos de esta práctica.

Ollama abstrae gran parte de estos procesos y proporciona una interfaz basada en API REST extremadamente sencilla de integrar desde Python.

Por tanto, la interacción con el modelo queda reducida a peticiones HTTP estándar sin necesidad de administrar internamente el ciclo completo de inferencia.

Reutilización de conocimientos y experiencia práctica

Otro factor relevante es que actualmente utilizo Ollama en proyectos personales y entornos de desarrollo propios relacionados con inteligencia artificial.

Trabajar con herramientas empleadas fuera del contexto académico permite reutilizar conocimientos previos y aproximar la implementación a escenarios reales de producción.

Esto facilita construir soluciones más cercanas a arquitecturas que posteriormente podrían escalar o reutilizarse en proyectos futuros.

Independencia y privacidad

La ejecución local aporta ventajas adicionales:

- ausencia de costes por uso;
- independencia de servicios externos;
- menor dependencia de cambios de APIs;
- privacidad de los datos procesados.

Toda la información permanece dentro del entorno local y no necesita enviarse a servicios externos para realizar inferencia.

Sustitución sencilla de modelos

La arquitectura implementada desacopla completamente la lógica de negocio del modelo utilizado.

Actualmente el proyecto emplea `gemma3`, pero el sistema permite sustituirlo fácilmente por alternativas como:

- Llama
- Mistral
- Qwen

sin modificar el resto del pipeline.

Por tanto, la decisión de utilizar Ollama no supone reemplazar las capacidades de Hugging Face, sino utilizar una capa de ejecución que simplifica considerablemente la integración y se adapta

mejor a los objetivos y contexto técnico del proyecto.

8. Dashboard interactivo

El proyecto incorpora una interfaz desarrollada mediante Streamlit para explorar resultados visualmente.

Funcionalidades principales:

- análisis de hashtags;
- nubes de palabras;
- evolución temporal;
- detección de bots;
- exploración de redes;
- análisis de comunidades;
- interacción mediante chat contextual con LLM.

Ejecución:

```
streamlit run dashboard/app.py
```

9. Instalación y despliegue

Crear entorno virtual

```
python -m venv venv
source venv/bin/activate

# Windows
venv\Scripts\activate
```

Instalar dependencias

```
pip install -r requirements.txt
```

Configurar variables de entorno

Crear archivo `.env`

```
RAPIDAPI_KEY=tu_clave
OLLAMA_URL=http://localhost:11434
LLM_MODEL=gemma3
```

Inicializar Ollama

```
ollama serve
ollama pull gemma3
```

10. Ejemplo completo de uso

```
from src import DataExtractor

ext = DataExtractor()

ext.load_data_api(
    query="#bitcoin",
    max_results=200
)

G = ext.build_interaction_graph()

metrics = ext.analyze_network(G)

prompt = ext.generate_prompt_from_network(
    G,
    metrics
)

ext.chat_local_llm(
    prompt=prompt,
    model="gemma3"
)
```

11. Limitaciones actuales

Las principales limitaciones identificadas son:

- limitaciones de la API gratuita utilizada;
- tamaño reducido de algunos datasets;
- tiempos elevados en CPU sin GPU;
- calidad dependiente del modelo local utilizado.

Estas restricciones podrían mitigarse mediante datasets más extensos o modelos de mayor tamaño.

12. Referencias y documentación

Referencias heredadas (AEC1–AEC2)

- 42 AI — fundamentos metodológicos del pipeline.

- RapidAPI — extracción de información en tiempo real.
- Pandas — manipulación de datos.
- NLTK — procesamiento lingüístico.
- TextBlob — análisis de sentimiento.
- spaCy — NLP avanzado.
- Gensim — modelado de tópicos.
- WordCloud — visualización textual.
- Matplotlib — visualización de datos.
- Streamlit — dashboards interactivos.

Referencias específicas AEC3

- NetworkX Documentation
- Network Science — Albert-László Barabási
- Community Detection Algorithms
- PageRank: The Anatomy of a Large Scale Hypertextual Web Search Engine
- python-louvain
- Ollama Documentation
- Gemma Technical Report
- Google Research Documentation